

Mobile Learning Application Development



Dart Programming Language Lecture 3

Aditya Rajbongshi
Assistant Professor

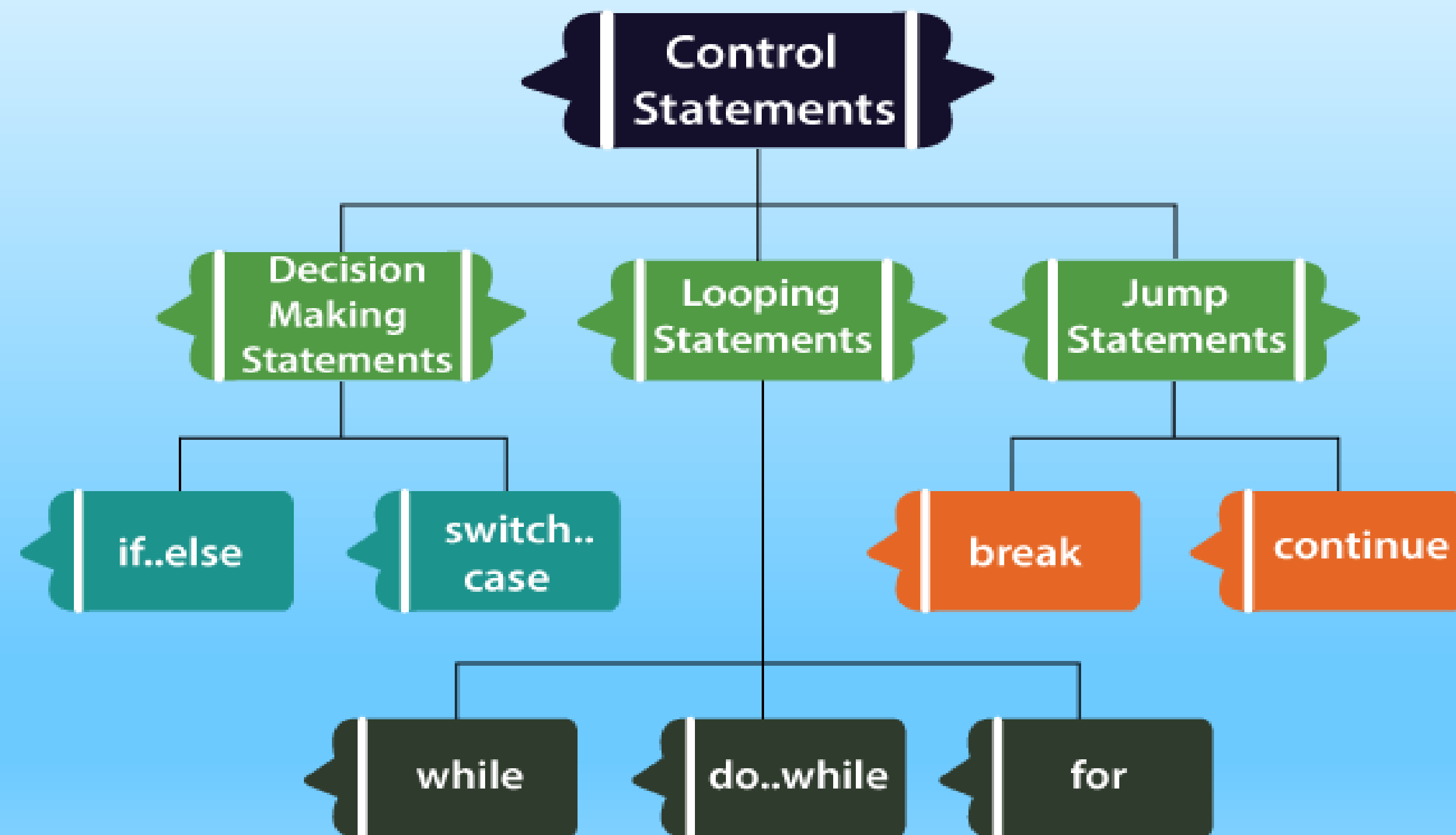
Dept. of Educational Technology and Engineering,
University of Frontier Technology, Bangladesh



Dart Control Flow Statement

The control statements or flow of control statements are used to **control the flow** of Dart program. In Dart, Control flow statement can be categorized mainly in **three** following ways.

- **Decision-making statements**
- **Looping statements**
- **Jump statements**





Dart Decision-Making Statements

The Decision-making statements allow us to determine which statement to execute based on the **test expression** at runtime. Dart provides following types of Decision-making statement.

- **If Statement**
- **If-else Statements**
- **If else if Statement**
- **Switch Case Statement**



If Statement

- If statement allows us to a block of code execute when the given condition returns true.

- Syntax -

```
If (condition) {  
    //statement(s)  
}
```

```
void main () {  
    var n = 35;  
    if (n<40) {  
        print("The number is smaller than 4  
0")  
    };  
}
```



else Statements

- In Dart, if-block is executed when the given condition is true. If the given condition is false, else-block is executed. The else block is associated with the if-block.

- Syntax -

```
if(condition) {  
    // statement(s);  
} else {  
    // statement(s);  
}
```

```
void main() {  
    var x = 20;  
    var y = 30;  
    print("if-else statement example");  
    if(x > y){  
        print("x is greater than y");  
    } else {  
        print("y is greater than x");  
    };  
}
```



if else-if Statement

- Dart if else-if statement provides the facility to check a set of test expressions and execute the different statements. It is used when we have to make a decision from more than two possibilities.

- Syntax -

```
if (condition1) {  
    }  
else if(condition2) {  
    } .  
.  
else {  
    }
```

```
void main() {  
    var marks = 74;  
    if(marks > 85)  
    {  
        print("Excellent");  
    }  
    else if(marks>75)  
    {  
        print("Very Good");  
    }  
    else if(marks>65)  
    {  
        print("Good");  
    }  
    else  
    {  
        print("Average");  
    }  
}
```




Switch Case Statement

- Dart Switch case statement is used to avoid the long chain of the if-else statement. It is the simplified form of nested if-else statement. The value of the variable compares with the multiple cases, and if a match is found, then it executes a block of statement associated with that particular case.

- Syntax -

```
switch( expression )  
{  
    case value-1:{  
        Block-1;  
        break;  
    case value-N:{  
        Block-N;  
        break;  
    default: {  
    }  
}
```

```
void main() {  
    int n = 2;  
    switch (n) {  
        case 1:  
            print("Value is 1");  
            break;  
        case 2:  
            print("Value is 2");  
            break;  
        case 3:  
            print("Value is 3");  
            break;  
        default:  
            print("Out of range");  
            break;  
    }  
}
```




Dart Looping statements

Dart Loop is used to run a block of code repetitively for a given number of times or until matches the specified condition. Loops are essential tools for any programming language. It is used to iterate the Dart **iterable such as list, map**, etc. and perform operations for **multiple times**. A loop can have **two parts** - a body of the **loop and control statements**. The main objective of the loop is to run the code multiple times. Dart supports the following type of loops.

- **Dart for loop**
- **Dart for...in loop**
- **Dart while loop**
- **Dart do-while loop**



Dart for loop

- The for loop is used when we know how many times a block of code will execute. It is quite same as the C for loop. The syntax is given below.

```
for(Initialization; condition; incr/decr) {  
    // loop body  
}
```

```
void main()  
{  
    int num = 1;  
    for(num; num<=10; num++) {  
        print(num);  
    }  
}
```



Dart for... in Loop

- The for...in loop is slightly different from the for loop. It only takes dart **object or expression** as an iterator and iterates the element **one at a time**. The value of the element is **bound to var**, which is and valid and available for the loop body. The syntax is given below.

```
for (var in expression) {  
  //statement(s)  
}
```

```
void main()  
{  
  var list1 = [10,20,30,40,50];  
  for(var i in list1) {  
    print(i);  
  }  
}
```



Dart while Loop

- The while loop executes a block of code until the given expression is false. It is more beneficial when we don't know the number of execution. The syntax is given below.

```
while(condition) {  
    // loop body  
}
```

```
void main()  
{  
    var a = 1;  
    var maxnum = 10;  
    while(a<maxnum){  
        print(a);  
        a = a+1;  
    }  
}
```




Dart do...while Loop

- The do...while loop is similar to the while loop but only difference is that, it executes the loop statement and then check the given condition. The syntax is given below.

```
do {  
    // loop body  
} while(condition);
```

```
void main()  
{  
    var a = 1;  
    var maxnum = 10;  
do  
{  
    print("The value is: ${a}");  
    a = a+1;  
}while(a<maxnum);  
}
```